Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Initiation à l'Algorithmique & Patrons de Conception de Parcours/Recherche

Synthèse exécutive : Ce cours établit le socle méthodologique fondamental de la conception algorithmique en BUT 1 Informatique. Avant toute écriture de pseudo-code, l'étudiant doit impérativement analyser la structure de données sous-jacente et son mécanisme d'accès (Direct, Séquentiel ou Dynamique). Le cours détaille et formalise trois modèles algorithmiques canoniques réutilisables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence. Une rigueur absolue est exigée sur le respect des bornes d'indices (de 0 à N-1) pour éviter les débordements de mémoire ('out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer une double condition d'arrêt pour tout algorithme de recherche.

1. Typologie & Mécanismes d'Accès aux Collections de Données

L'étape zéro de toute conception algorithmique consiste à analyser le contrat d'interface offert par la structure de données que l'on souhaite manipuler. Si la collection propose un accès direct (comme un tableau statique d'entiers ou une chaîne de caractères), chaque case mémoire est immédiatement adressable par son indice i. En revanche, un flux séquentiel (tel qu'un fichier texte brut, un flux CSV lu ligne par ligne ou une liste chaînée simple) n'expose que trois opérations primitives : (1) se positionner au début, (2) avancer d'un pas vers l'élément suivant, et (3) tester la fin de flux. Tenter d'utiliser une boucle itérative indicée avec un accès direct de type 'T[i]' sur un flux séquentiel constitue une erreur conceptuelle critique qui invalide totalement l'algorithme.

- Accès direct : Capacité d'atteindre instantanément n'importe quel élément via son indice ou sa clé (ex: T[i]) en temps constant O(1), sans parcourir les éléments précédents.
- Accès séquentiel : Contrainte imposant de démarrer au début de la structure et de progresser pas à pas (début, suivant, fin) sans possibilité de saut direct.
- Accès dynamique : Structure hybride proposant à la fois des primitives d'accès séquentiel et des mécanismes d'indexation directe.
- Incompatibilité structurelle : Interdiction absolue d'appliquer un modèle algorithmique nécessitant l'accès direct sur un flux purement séquentiel.

Cas concret enseignant : *Le professeur insiste sur un piège classique en évaluation : tenter d'écrire une notation par indice 'T[i]' sur un fichier CSV brut ou un fichier texte ouvert. Comme le système d'exploitation ne fournit qu'un curseur séquentiel avance-ligne, un tel code est sémantiquement faux et conduit directement à la note zéro.*

2. Premier Modèle : Parcours Complet avec Traitement Systématique

Ce modèle algorithmique s'applique dès lors qu'aucune exclusion n'est permise entre deux bornes définies. Il répond à des problématiques telles que : afficher toutes les valeurs d'un tableau, initialiser chaque case à zéro, ou convertir chaque caractère d'une chaîne en majuscule. Schéma canonique : Pour i allant de BorneDébut à BorneFin Faire Traiter(Collection[i]) Fin Pour En notation standard (indexation à zéro), pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Le traitement est exécuté de manière inconditionnelle à chaque itération.

- Exhaustivité : 100% des éléments délimités entre une borne initiale et une borne finale subissent le traitement.
- Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' (car le nombre d'itérations est connu avant le démarrage : Fin - Début + 1).
- Gestion stricte des bornes : Pour un tableau de N éléments indicé à partir de 0, l'indice varie strictement de 0 à N - 1.
- Risque critique 'Out of Bounds' : Faire varier l'indice jusqu'à N provoque un débordement mémoire fatal.

Cas concret enseignant : *Exemple d'illustration du prof : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un plantage immédiat (crash out of bounds) à l'exécution lors de la tentative d'accès à la case T[N] qui n'existe pas.*

3. Second Modèle : Parcours Complet avec Traitement Conditionné

Le modèle du parcours complet conditionné conserve l'exploration exhaustive du modèle 1, mais encapsule l'action de traitement dans une condition logique. Il répond aux problèmes de comptage sélectif, de filtrage d'affichage ou d'accumulation conditionnelle. Schéma canonique : Pour i allant de BorneDébut à BorneFin Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour Si la condition implique d'analyser l'élément précédent (ex: T[i] > T[i-1]), les bornes de la boucle doivent être réajustées (ex: démarrer i à 1) sous peine de provoquer un accès hors limites sur T[-1].

- Parcours exhaustif : Tous les éléments de la zone sont inspectés de la borne de début à la borne de fin.
- Traitement sélectif : L'action de traitement n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).
- Architecture algorithmique : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.
- Conditions complexes : Le prédicat peut combiner plusieurs critères (ET, OU) ou comparer l'élément courant à ses voisins (i-1 ou i+1).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine : 'Chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet de la chaîne), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).'*

4. Troisième Modèle : Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, la recherche de première occurrence s'interrompt dès que la cible est localisée. L'informaticien doit adopter une posture pessimiste : poser au départ que l'élément est absent (`trouve <- Faux`). Tout algorithme de recherche valide DOIT comporter deux conditions d'arrêt : 1. La recherche est fructueuse (trouve = Vrai). 2. La recherche a échoué après exploration complète de la structure. Comparatif des variantes : - Variante 1 : Utilise une sortie anticipée. Elle garantit que l'indice 'i' pointe précisément sur l'emplacement de l'élément trouvé. - Variante 2 : Utilise une boucle `Tant que (i < N et NON trouve)`. Si l'élément est trouvé, 'trouve' passe à Vrai, mais 'i' est incrémenté une dernière fois, ce qui nécessite un réajustement d'indice (i - 1) si l'on a besoin de la position exacte.

- Pessimisme algorithmique : Initialisation obligatoire d'un indicateur booléen 'trouve' à Faux avant d'entamer l'exploration.
- Itération non bornée : Utilisation d'une boucle 'Tant que' car le nombre d'itérations est imprévisible.
- Double condition d'arrêt obligatoire : (1) Succès (trouve = Vrai), (2) Échec par épuisement de la collection (Fin de structure atteinte).
- Variante 1 (Rupture) : Interrompt la boucle dès la découverte ; conserve la variable d'indice 'i' exactement positionnée sur l'élément trouvé.
- Variante 2 (Tant que global) : Regroupe la double condition en tête ('Tant que i < N et NON trouve') ; très propre syntaxiquement mais incrémente 'i' une ultime fois avant de sortir.

Cas concret enseignant : *Devise du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans votre collection, et ne faites basculer votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il rappelle qu'écrire un algorithme de recherche avec une seule condition d'arrêt entraîne la note zéro en partiel.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systematic sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique offre à la fois des primitives d'accès séquentiel et d'accès direct. On peut donc parfaitement appliquer le modèle du parcours complet systématique en exploitant ses mécanismes d'accès direct par indice.

Question étudiant : Pourquoi s'embêter à apprendre deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce qu'elles répondent à des besoins d'implémentation différents. La Variante 1 (avec rupture) est idéale si vous avez besoin de récupérer la position exacte (l'indice) de l'élément trouvé, car la boucle s'arrête net sur la case concernée. La Variante 2 (Tant que global) est plus élégante et standard syntaxiquement (test unique en tête), mais comme l'indice avance d'un pas supplémentaire avant l'évaluation de sortie, elle demande un petit calcul de réajustement d'indice si l'on souhaite réutiliser la position.

Question étudiant : Comment fait-on si la condition dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut réajuster les bornes de la boucle 'Pour'. Par exemple, si votre test compare l'élément courant T[i] avec l'élément précédent T[i-1], la boucle ne doit pas commencer à 0 mais à 1. Sinon, lors de la première itération à i=0, l'algorithme tenterait d'accéder à T[-1], ce qui déclencherait une erreur fatale d'indice hors limites.

Question étudiant : Si l'on souhaite récupérer TOUS les indices correspondant à une valeur (par exemple trouver toutes les occurrences du nombre 5 dans un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Attention au piège ! Il s'agit alors d'un 'Parcours complet avec traitement conditionné' et NON d'une recherche de première occurrence. En effet, comme il faut trouver TOUTES les occurrences, on ne doit pas s'arrêter au premier 5 trouvé : on doit obligatoirement parcourir le tableau jusqu'à la dernière case et afficher ou stocker l'indice chaque fois que T[i] est égal à 5.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et choix du modèle. On doit inspecter la totalité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle canonique est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes d'indices. En indexation standard (commençant à 0), pour N éléments, les indices s'étendent de 0 à N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure de contrôle itérative. Nombre d'itérations connu à l'avance -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Filtre logique et action de traitement. À chaque tour de boucle, évaluer 'Si T[i] > 0 Alors' et déclencher l'action 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit un parcours exhaustif sans risque de débordement mémoire (l'indice i s'arrête strictement à N-1) et filtre sélectivement les valeurs strictly positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on impérativement employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence. On utilise ce modèle dès qu'il s'agit de vérifier l'existence d'au moins un élément répondant à un critère et d'interrompre le parcours dès qu'il est trouvé.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce qu'en informatique standard, l'indexation des tableaux commence à 0. Un tableau de N éléments contient donc des cases numérotées de 0 à N - 1. Accéder à la case N tente d'atteindre une zone mémoire non attribuée au tableau et génère une erreur fatale de débordement d'indice (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Un algorithme de recherche valide doit obligatoirement comporter au moins 2 conditions d'arrêt : (1) Le succès de la recherche (l'élément recherché a été trouvé), et (2) L'échec par épuisement de la structure (tous les éléments ont été explorés sans succès).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765